#1

1. Inserting at the beginning of a list takes $O(n)$, and while loop takes $O(n)$.
   ∴ The worst case running time : $1+2+3+\ldots+n = \frac{n(n+1)}{2} = O(n^2)$

2. Append takes armortized $O(1)$, and while loop runs n times
   $\underbrace{1+1+\ldots+1}_{n} = n$
   ∴ The worst case running time is $O(n)$.

#2 c) 1. append and pop takes armortized cost of constant runtime.
For n operations, the cost append/pop would be amortized $O(1)$ per operation, and the total cost would be $\sum_{i=0}^{n} 1 = \underbrace{1+1+1+\ldots+1}_{n \text{ times}} = n = O(n)$

For $2n$ operations, the cost append/pop would be amortized $O(1)$ per operation, and the total cost would be $\sum_{i=0}^{2n} 1 = \underbrace{1+1+\ldots+1}_{2n \text{ times}} = 2n$, which is still be $O(n)$ since the constant is dropped.

2. Let array be an empty array. suppose that append() is called to the array until the array has to double the size and that pop() is called to the array until the array has to shrink the size. Define n to be the operations.
The cost for append will be $1+2+4+\ldots+n = \frac{n(n+1)}{2} = \Omega(n^2)$
the cost of pop is $(n-1)+(n-2)+(n-3)+\ldots+1 = \frac{(n-1)(n)}{2} = \Omega(n^2)$.

#3 b) The worst-case running time of my implementation is $O(n)$.
Since my implementation use two for loops that parallel with the inside loop that takes $O(1)$ time. Each loop runs $O(n)$ time
Creating counter list takes $O(n)$ because it creates a list of zeros with a size of the original list minus 1.
Thus, the total worst-case running time is $O(n+n+n) = O(3n) = O(n)$

#4 a) The remove(value) method removing the first occurence of value takes $O(n)$ time because it goes through the entire list until found value and shift the subsequent elements.
The while loop iterates until there is no element in the list, and the size of list decreases by 1 in each iteration due to the remove(value) method. This cause time complexity to be:
$n + (n-1) + (n-2) + \ldots + 1 = O(n \times n) = O(n^2)$

c) My implementation has the worst-case runtime of $O(n)$.

The while loop runs $O(n)$ because it goes through the list, and inside the loop is $O(1)$. ($O(1)$ per if, $O(1)$ per swap)

The for loop also runs $O(n)$ because the inside

the loop, pop() also takes $O(1)$ time.

These loops are parallel, meaning that the total worst case runtime is $O(n+n) = O(2n) = O(n)$